

Complete C++ Programming Language Guide

From Basics to Advanced

A Comprehensive Learning Resource for Beginners

Table of Contents

1. Introduction to C++
 2. Basics
 3. Data Types & Variables
 4. Operators
 5. Control Flow
 6. Functions
 7. Arrays & Pointers
 8. Object-Oriented Programming
 9. Advanced Concepts
 10. Standard Template Library (STL)
- Practice Problems
- Resources & Quick Reference

1. INTRODUCTION TO C++

What is C++?

C++ is a **general-purpose, object-oriented programming language** created by Bjarne Stroustrup in 1985. It extends C with object-oriented features while maintaining efficiency and performance.

Key Features

- **Fast & Efficient:** Compiled language, close to hardware
- **Object-Oriented:** Supports classes, inheritance, polymorphism
- **Generic Programming:** Templates for reusable code
- **Low-level Access:** Pointers and direct memory manipulation
- **Rich Standard Library:** STL provides powerful data structures

Why Learn C++?

- Used in game development, system software, competitive programming
- High performance required applications
- Interview preparation (popular in tech interviews)
- Foundation for understanding computer science concepts

Setting Up C++

Compiler Options:

- GCC (Linux/Mac): `g++ program.cpp -o program`
- Clang (All platforms)
- MSVC (Visual Studio - Windows)
- Online: OnlineGDB, Replit, LeetCode

2. BASICS

Your First Program

```
#include <iostream>      // Input/Output library
using namespace std;    // Use standard namespace

int main() {
    std::cout <<< "Hello, World!" <<< std::endl;
    return 0;           // Program ends successfully
}
```

Understanding the Components

- **#include:** Includes a library/header file
- **iostream:** Standard library for input/output
- **using namespace std:** Avoids writing std:: before cout, cin
- **main():** Entry point of every C++ program
- **cout:** Output stream (print to console)
- **cin:** Input stream (read from keyboard)
- **endl or \n:** Newline character

Input & Output Example

```
#include <iostream>
using namespace std;

int main() {
    int age;
    string name;

    cout <<< "Enter your name: ";
    cin >>> name;

    cout <<< "Enter your age: ";
    cin >>> age;

    cout <<< "Name: " <<< name <<< ", Age: " <<< age <<< endl;
    return 0;
}
```

3. DATA TYPES & VARIABLES

Primitive Data Types

Data Type	Size	Range	Example
int	4 bytes	-2^{31} to $2^{31}-1$	int x = 10;
float	4 bytes	$\sim \pm 3.4e\pm 38$	float f = 3.14f;
double	8 bytes	$\sim \pm 1.7e\pm 308$	double d = 3.14159;
char	1 byte	-128 to 127	char c = 'A';
bool	1 byte	true/false	bool flag = true;
long	8 bytes	Large integers	long x = 100000L;

Variable Declaration & Initialization

```
int x;           // Declaration (uninitialized)
int y = 10;      // Declaration with initialization
int a = 5, b = 10; // Multiple variables
const int MAX = 100; // Constant variable
```

Type Conversion

```
// Implicit conversion (automatic)
int x = 10;
double d = x; // int converted to double

// Explicit conversion (casting)
int a = (int)3.14; // C-style cast
int b = static_cast<int>(3.14); // C++ style cast
```

4. OPERATORS

Arithmetic Operators

```
int a = 10, b = 3;  
a + b; // Addition = 13  
a - b; // Subtraction = 7  
a * b; // Multiplication = 30  
a / b; // Division = 3  
a % b; // Modulus (remainder) = 1
```

Comparison Operators

```
a == b; // Equal to  
a != b; // Not equal to  
a > b; // Greater than  
a < b; // Less than  
a >= b; // Greater than or equal  
a <= b; // Less than or equal
```

Logical Operators

```
a && b; // AND: true if both true  
a || b; // OR: true if either true  
!a; // NOT: inverts boolean value
```

Ternary Operator

```
int age = 20;  
string status = (age >= 18) ? "Adult" : "Minor";
```

5. CONTROL FLOW

If-Else Statements

```
int age = 20;
if (age >= 18) {
    cout <<< "Adult" <<< endl;
}
else if (age >= 13) {
    cout <<< "Teenager" <<< endl;
}
else {
    cout <<< "Child" <<< endl;
}
```

For Loop

```
for (int i = 1; i <= 5; i++) {
    cout <<< i <<< " ";
}
// Output: 1 2 3 4 5
```

While Loop

```
int i = 1;
while (i <= 5) {
    cout <<< i <<< " ";
    i++;
}
// Output: 1 2 3 4 5
```

Range-based For Loop (C++11)

```
int arr[] = {1, 2, 3, 4, 5};
for (int num : arr) {
    cout <<< num <<< " ";
}
// Output: 1 2 3 4 5
```

6. FUNCTIONS

Function Declaration & Definition

```
// Function prototype (declaration)
int add(int a, int b);

// Function definition
int add(int a, int b) {
    return a + b;
}

// Function call
int result = add(5, 3); // result = 8
```

Pass by Value vs Pass by Reference

```
// Pass by Value (creates a copy)
void incrementByValue(int x) { x++; }

// Pass by Reference (modifies original)
void incrementByRef(int &x) { x++; }

int num = 10;
incrementByValue(num); // num is still 10
incrementByRef(num);   // num is now 11
```

Recursive Functions

```
int factorial(int n) {
    if (n <= 1) {
        return 1; // Base case
    }
    return n * factorial(n - 1); // Recursive case
}

factorial(5); // Returns 120
```

7. ARRAYS & POINTERS

Arrays

```
// Declaration
int arr[5];                // Array of 5 integers

// Initialization
int arr[5] = {1, 2, 3, 4, 5};
int arr[] = {10, 20, 30};  // Size inferred

// Accessing elements
cout <<< arr[0] <<< endl;    // First element

// 2D Arrays
int matrix[3][3] = {
    {1, 2, 3}, {4, 5, 6}, {7, 8, 9}
};
```

Pointers

A pointer stores the memory address of another variable.

```
int x = 10;
int *ptr = &x;            // & = address-of operator

cout <<< ptr <<< endl;    // Prints memory address
cout <<< *ptr <<< endl;   // Prints value (10)

// Modifying through pointer
*ptr = 20;                // x is now 20
```

Dynamic Memory Allocation

```
// Single integer
int *ptr = new int;
*ptr = 10;
delete ptr;               // Free memory

// Array allocation
int *arr = new int[5];
arr[0] = 10;
delete[] arr;             // Free array memory
```


8. OBJECT-ORIENTED PROGRAMMING

Classes & Objects

```
class Car {
private:
    string brand;
    int speed;

public:
    // Constructor
    Car(string b, int s) : brand(b), speed(s) {}

    // Getter methods
    string getBrand() { return brand; }
    int getSpeed() { return speed; }

    // Setter methods
    void setSpeed(int s) {
        if (s >= 0) speed = s;
    }
};

int main() {
    Car myCar("Toyota", 100);
    myCar.setSpeed(150);
    return 0;
}
```

Inheritance

```
class Animal {
public:
    void eat() {
        cout <<< "Animal is eating" <<< endl;
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout <<< "Dog is barking" <<< endl;
    }
};

Dog dog;
dog.eat(); // Inherited from Animal
dog.bark(); // Own method
```

Polymorphism (Virtual Functions)

```
class Animal {
public:
    virtual void sound() {
        cout <<< "Generic sound" <<< endl;
    }
};

class Dog : public Animal {
public:
    void sound() override {
        cout <<< "Woof! Woof!" <<< endl;
    }
};

int main() {
    Animal *animal = new Dog();
    animal->sound(); // Output: Woof! Woof!
    delete animal;
    return 0;
}
```

9. STANDARD TEMPLATE LIBRARY (STL)

Vectors (Dynamic Arrays)

```
#include <vector>
using namespace std;

vector<int> v; // Empty vector
vector<int> v = {1, 2, 3}; // Initialization

v.push_back(4); // Add element
v.pop_back(); // Remove last
cout << v[0] << endl; // Access element
cout << v.size() << endl; // Get size
```

Maps (Key-Value Pairs)

```
#include <map>
map<string, int> ages;

ages["John"] = 30;
ages["Jane"] = 25;

cout << ages["John"] << endl; // 30
ages.erase("Jane");

for (auto &pair : ages) {
    cout << pair.first << ": " << pair.second << endl;
}
```

Algorithms

```
#include <algorithm>
vector<int> v = {5, 2, 8, 1, 9};

sort(v.begin(), v.end()); // {1, 2, 5, 8, 9}
reverse(v.begin(), v.end()); // Reverse

auto it = find(v.begin(), v.end(), 5);
if (it != v.end()) {
    cout << "Found!" << endl;
}

int cnt = count(v.begin(), v.end(), 5); // Count occurrences
```

10. PRACTICE PROBLEMS

Beginner Level

1. Sum of two numbers
2. Check if number is prime
3. Reverse a string
4. Check palindrome
5. Calculate factorial

Intermediate Level

1. Binary search implementation
2. Merge two sorted arrays
3. Find duplicates in array
4. Implement bubble sort
5. Create a simple student management system

Advanced Level

1. Build a graph and implement DFS/BFS
2. Implement AVL tree
3. Design a file system
4. Create a cache system (LRU Cache)
5. Implement trie data structure

RESOURCES FOR FURTHER LEARNING

Online Platforms

1. **GeeksforGeeks** (geeksforgeeks.org) - Comprehensive tutorials
2. **LeetCode** (leetcode.com) - Practice coding problems
3. **HackerRank** (hackerrank.com) - Structured learning
4. **Codeforces** (codeforces.com) - Competitive programming
5. **YouTube Channels:**
 - Cherno C++ Series
 - Brackeys
 - Errichto

Recommended Books

1. "A Tour of C++" by Bjarne Stroustrup
2. "The C++ Programming Language" (The Bible)
3. "Effective C++" by Scott Meyers
4. "C++ Primer Plus" by Stephen Prata

Practice Tips

- Write code daily, even small snippets
- Understand concepts, don't memorize
- Debug your own code
- Read others' code and learn patterns
- Participate in competitive programming
- Build small projects to apply concepts

QUICK REFERENCE

Compilation & Execution

g++ -o program program.cpp	# Compile
./program	# Execute (Linux/Mac)
program.exe	# Execute (Windows)

Common Syntax

```
// Variable declaration
int x = 10;
int *ptr = &x;
int &ref = x;

// Function
return_type function_name(parameters) { }
```

```
// Class
class ClassName { public: private: protected: };
```

```
// Loops
for(;;) { }
while(condition) { }
do { } while(condition);
```

```
// Conditional
if() { } else if() { } else { }
switch(x) { case 1: break; default: }
```

CONCLUSION

C++ is a powerful language that requires patience and consistent practice. Start with the fundamentals of the language, gradually progress to object-oriented programming principles, and finally explore advanced concepts like templates and the Standard Template Library (STL). The key to mastering C++ is to understand the "why" behind each concept rather than simply memorizing syntax. Apply what you learn through practical coding exercises, build small projects, and gradually increase the complexity of your programs. Remember that every expert programmer started as a beginner. Consistency and practice are your best tools for success. Write code every day, debug your own mistakes, and don't be afraid to experiment with new concepts. Happy coding, and best of luck on your C++ learning journey!

